

FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO - FECAP

Bacharelado em Ciência da Computação

Projeto Interdisciplinar: IoT e Robótica – Professor(a) João Trencher

Estudantes:

- André Gregório dos Santos – RA: 24026489
- Guilherme Reis Fogolin de Godoy – RA: 24026241
- Pedro Henrique Nascimento Lemos – RA: 23025380
- Yan Ramos Cezareto – RA: 24026005

Turma: 6NACOMP / 6NACOMP_S

Smart Tennis Arena: controle da máquina lançadora de bolas por voz

Apresentação do projeto

A Smart Tennis Arena é o projeto interdisciplinar do 6º semestre de Ciência da Computação da FECAP, feito em parceria com a Smart Tennis. O **núcleo do projeto é o controle da máquina lançadora de bolas por voz**: o praticante fala e os motores ligam ou desligam, sem largar a raquete. Em volta desse núcleo está a plataforma que o enunciado cobra, com aplicativo, site de autoria e API, que dá contexto de treino ao disparo.

O grupo já construiu o protótipo físico. O **NockHug** é uma máquina lançadora de bolas de ping-pong controlada por voz. Um motor lançador de 14 000 RPM gira uma roda na ponta de um cano de PVC e arremessa a bola por atrito; um motor alimentador de 20 RPM gira a base que posiciona as bolinhas na entrada do cano. O ESP32 DevKit V1 gera dois sinais PWM (GPIO25 e GPIO26), e cada sinal comanda um motor por um MOSFET IRLZ44N. A página que o próprio ESP32 serve reconhece os comandos no navegador do celular e chama os endpoints `/ligar`, `/desligar` e `/status`.

Além de protótipo, o NockHug é a camada de adaptação que representa os comandos da máquina de tênis comercial. O enunciado autoriza essa função: a integração física com uma máquina real só pode usar interface documentada e autorizada, e onde não há interface oficial ele manda usar um simulador. A Seção 1.4 delimita até onde essa representação vai.

As seções **1, 2, 4, 5, 6 e 7** correspondem, na ordem, aos seis itens exigidos pelo enunciado: descrição do problema e escopo, diagrama de blocos, fluxo de dados, requisitos, máquina de estados e Product Backlog. A **seção 3** aprofunda o núcleo do projeto, que é o controle por voz.

Índice

Apresentação do projeto	1
1 Descrição técnica do problema e delimitação do escopo	3
2 Diagrama de blocos do sistema	5
3 O controle por voz	7
4 Fluxo de dados	9
5 Requisitos funcionais e não funcionais	11
6 Máquina de estados do sistema	13
7 Product Backlog	16
8 Uso de inteligência artificial	20

1 Descrição técnica do problema e delimitação do escopo

1.1 O problema

Máquinas lançadoras como a Spinshot Plus-2 resolvem a parte mecânica: velocidade, rotação, direção, altura, oscilação e intervalo são configuráveis. O que não está resolvido é o controle e o que acontece em volta do disparo. A configuração é manual e local, feita na própria máquina ou num controle dedicado, e não existe relação entre a definição do exercício e o desempenho medido.

A Smart Tennis Arena ataca as duas pontas. O **controle por voz** resolve o acesso à máquina durante o treino, e a **plataforma** transforma cada sessão em dado: o treino vira um objeto com autor, versão, regras de pontuação e histórico.

1.2 Por que o controle por voz

Quem opera uma lançadora está em quadra, com a raquete em uma das mãos e as bolas no bolso. Para interromper o disparo, essa pessoa precisa largar o que está fazendo, atravessar a quadra e alcançar o celular ou o painel da máquina. O deslocamento quebra o ritmo do treino, e é justamente entre as séries que a maioria desiste de ajustar qualquer coisa.

O controle por voz elimina o deslocamento: o comando sai de onde o praticante já está. Há um segundo motivo, de acessibilidade. Mão suada, luva, raquete ocupada e sol direto na tela são todos obstáculos ao toque, e nenhum deles afeta a fala. Por isso a voz não é uma funcionalidade acessória neste projeto: é a interface principal, e é ela que sustenta as decisões de arquitetura, de segurança e de modelagem das seções seguintes.

1.3 Escopo do MVP

O escopo abaixo cobre o cenário de aceitação do enunciado de ponta a ponta: o autor cria e publica um treino no site; o aplicativo obtém o treino pela API; o praticante conecta o dispositivo e inicia a sessão; sensores e atuadores geram eventos e feedback; a máquina ou o simulador executa a sequência; o sistema calcula a pontuação, permite parada segura, encerra a sessão e sincroniza os resultados.

Entra no MVP:

- **controle por voz** de ligar, desligar, pausar, retomar e emergência, com validação do comando na frase transcrita;
- aplicativo mobile para preparar, executar e acompanhar a sessão, com modo adulto e modo criança;
- site para o autor criar, validar, versionar e publicar treinos;
- API REST com contrato OpenAPI para treinos, sessões e telemetria;

- dispositivo IoT com os dois canais de acionamento já montados e a geração de eventos de etapa;
- pontuação gamificada por regras declaradas, e parada segura como estado absorvente;
- operação offline do treino sincronizado, com fila de reenvio.

Fica fora do MVP:

- cobrança e remuneração de autores, que o RF20 trata apenas como metadados;
- integração física com máquina comercial, por restrição do enunciado;
- visão computacional, sensores corporais e dados fisiológicos;
- reconhecimento de fala embarcado no aparelho, que fica como caminho de saída para a dependência de rede (Seção 3.4).

1.4 A camada de adaptação

Como não há interface documentada para a máquina de tênis comercial, o NockHug faz o papel de simulador. O Quadro 1 declara o que ele representa e o que fica de fora. A coluna da direita é o roteiro de integração futura: cada linha só poderá ser comandada quando existir interface oficial.

Quadro 1 — Cobertura da camada de adaptação em relação à máquina de tênis real.

Capacidade da máquina real	Como o NockHug representa	Situação
Disparo contínuo de bolas	Motor lançador M1 (14 000 RPM) por PWM em GPIO25.	Representado
Alimentação sequencial	Motor alimentador M2 (20 RPM) por PWM em GPIO26.	Representado
Ligar e desligar a máquina	Por voz, com rampa de aceleração ao ligar e parada imediata ao desligar.	Representado
Velocidade da bola	Duty cycle do PWM, sem correspondência em km/h.	Fora do MVP
Rotação (topspin / slice)	Não há mecanismo de rotação.	Fora do MVP
Direção e oscilação	Não há eixo de varredura horizontal.	Fora do MVP
Altura da bola	Não há ajuste de elevação do cano.	Fora do MVP
Intervalo entre bolas	Definido no firmware, não pelo treino.	Parcial
Medição de acerto e erro	Eventos de etapa gerados pelo firmware, sem sensor físico.	Simulado
Confirmação de estado	GET /status devolve LIGADO ou DESLIGADO.	Representado

O sistema demonstra o fluxo completo de dados e comandos, como o enunciado permite para o MVP, sem afirmar que mede o que ainda não mede.

1.5 O que já existe

Boa parte da fundação está pronta. O circuito de acionamento está montado e medido em protoboard. O firmware do ESP32 sobe a rampa de aceleração, cria o Access Point com senha própria e exige token compartilhado em `/ligar` e `/desligar`, respondendo HTTP 401 sem ele. O controle por voz funciona no navegador, com reconhecimento em pt-BR e prioridade para o comando de desligar. A modelagem formal do controle por voz em dois autômatos finitos e o relatório de vulnerabilidades também estão entregues. O aplicativo, o site, a API e o banco ainda não têm código, e não há sensor físico: os eventos de etapa são gerados pelo firmware, como registra o Quadro 1.

2 Diagrama de blocos do sistema

A Figura 1 organiza a solução em quatro camadas (nuvem, serviços, aplicações e campo) e mostra por onde cada uma conversa com a vizinha. A faixa de campo traz a cadeia do controle por voz, do microfone até o motor, que é o caminho aprofundado na Seção 3.

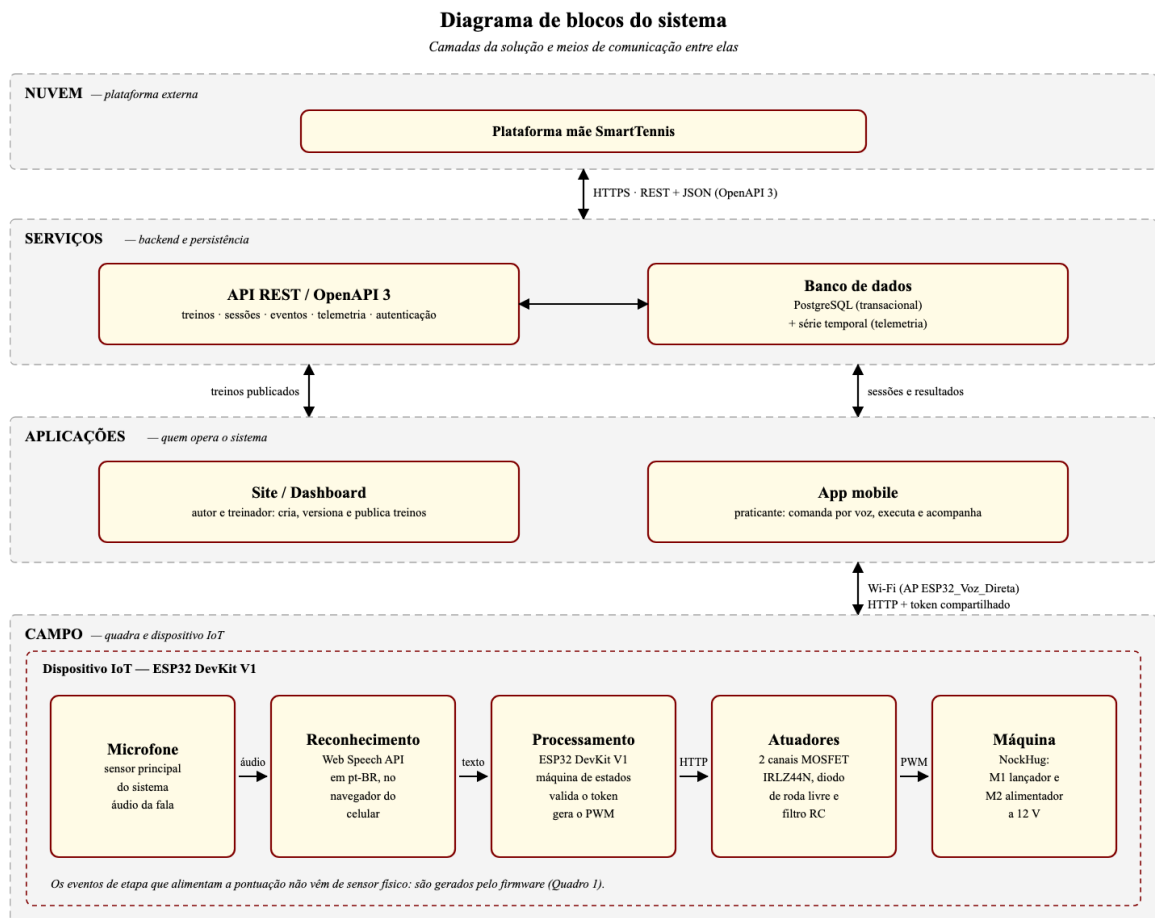


Figura 1 — Diagrama de blocos do sistema. Fonte: Autoria própria.

2.1 Blocos e comunicação

Quadro 2 — Blocos do sistema, responsabilidade e enlace de comunicação.

Bloco	Responsabilidade	Comunicação
Plataforma mãe	Sistema externo do parceiro, de onde vêm os treinos e para onde voltam resultados e telemetria.	HTTPS, REST, JSON
Site / dashboard	O autor cria, valida, versiona e publica treinos, e analisa os resultados.	HTTPS com sessão do usuário
App mobile	O praticante comanda por voz, prepara e executa o treino, e consulta resultado e histórico.	HTTPS com sessão e cache local
API REST / OpenAPI 3	Ponto único de integração, com autenticação e autorização por perfil.	HTTPS, REST, JSON
Banco de dados	PostgreSQL para contas, treinos, sessões e resultados, com série temporal para telemetria.	Interno à API
Microfone	Sensor principal do sistema. Captura o áudio da fala no celular do praticante.	Áudio, dentro do aparelho
Reconhecimento de fala	Transcreve o áudio em pt-BR e valida o comando na frase. Roda no navegador, não no ESP32.	Web Speech API
Processamento (ESP32)	Valida o token, controla a máquina de estados e calcula o duty cycle dos dois canais.	Wi-Fi (Access Point), HTTP com token
Atuadores	Dois canais com MOSFET IRLZ44N, diodo de roda livre e filtro, chaveando a cerca de 1 kHz.	PWM em GPIO25 e GPIO26
Máquina lançadora	Motor M1 (lançador) e M2 (alimentador), ambos de 12 V.	12 V chaveado

A decisão de arquitetura mais importante do projeto está na separação entre o reconhecimento e o processamento. O ESP32 nunca vê áudio: ele recebe requisições já decididas. Isso mantém o firmware pequeno e deixa a troca do motor de reconhecimento restrita ao aplicativo, sem tocar no embarcado. O custo dessa escolha aparece na Seção 3.4.

2.2 Pilha tecnológica

O critério principal é **uma linguagem só** em todo o software. Com React Native no mobile, Next.js no site e Node.js na API, tudo é TypeScript, o que reduz a troca de contexto de uma equipe pequena e permite compartilhar os tipos do contrato entre cliente e servidor.

Quadro 3 — Pilha tecnológica e justificativa.

Camada	Tecnologia	Por que
Aplicativo	React Native com TypeScript	Um código para Android e iOS. O modo offline usa armazenamento local, e a comunicação com o dispositivo é por Wi-Fi e HTTP, o que evita a fragilidade do suporte a Bluetooth Low Energy no React Native.

Camada	Tecnologia	Por que
Site e API	Next.js e Node.js, com TypeScript	Contrato OpenAPI gerado a partir do código, o que mantém a documentação do RF18 sincronizada com a implementação.
Banco de dados	PostgreSQL com extensão de séries temporais	Um banco relacional para os dados transacionais, com a mesma instância servindo a telemetria.
Embarcado	ESP32 DevKit V1, C++ com Arduino	Já é a plataforma do protótipo, com Wi-Fi integrado e PWM em hardware.
Reconhecimento de fala	Web Speech API, no navegador	Já está implementado e funcionando, sem custo de licença. A dependência que ele cria está declarada na Seção 3.4, com o caminho de saída para a Entrega 2.
Implantação	Contêineres em nuvem, variáveis de ambiente e monitoramento básico	Atende ao RNF de observabilidade e permite reprovisionar sem depender da máquina de um integrante.

3 O controle por voz

Esta seção aprofunda o núcleo do projeto. A Figura 2 mostra o caminho completo da fala até o motor e onde cada etapa roda.

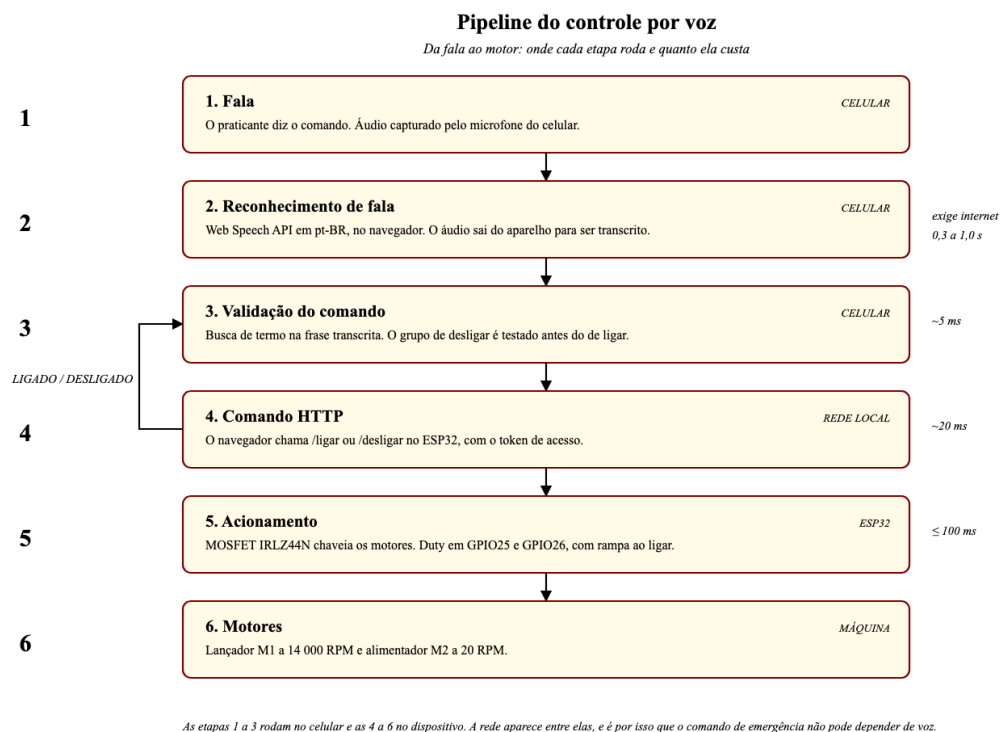


Figura 2 — Pipeline do controle por voz, da fala ao motor. Fonte: Autoria própria.

3.1 O caminho da fala ao motor

O microfone do celular capta o áudio e a Web Speech API o transcreve em português. O texto volta para a página, que procura nele um termo de comando. Se encontrar, o navegador chama o endpoint correspondente no ESP32, com o token de acesso. O ESP32 valida o token, consulta a máquina de estados e aciona os motores por PWM. A resposta volta para a página, que atualiza o status exibido.

Três decisões moldam esse caminho. A primeira é que **a validação acontece no navegador**, e não no ESP32: o microcontrolador só recebe requisições já decididas. A segunda é que **a busca é por termo dentro da frase**, e não por frase exata, o que permite falar naturalmente ("pode desligar agora") em vez de palavras isoladas. A terceira é a **prioridade do desligar**: o grupo de termos de desligar é testado antes do de ligar, porque "desligar" contém "ligar". Sem essa ordem, todo pedido de desligar seria lido como pedido de ligar.

Uma quarta característica do caminho atual merece registro, porque afeta a segurança. A rampa de aceleração roda em laço fechado no firmware, 51 passos de 15 ms, e durante esses 765 ms o servidor HTTP não atende ninguém. Um "desliga" dito nesse intervalo só é processado quando a rampa termina. O firmware da Entrega 2 deve passar a rampa para o `loop()`, para que o desligar seja atendido a qualquer momento.

3.2 Vocabulário

O Quadro 4 reúne os comandos. Os dois primeiros já estão implementados no protótipo; pausar, retomar e a palavra de emergência entram na Entrega 2, e é a coluna da direita que amarra cada um à trava de segurança correspondente.

Quadro 4 — Vocabulário de voz, efeito de cada comando e onde ele é aceito.

Comando	Termos aceitos	O que faz	Onde é aceito
Ligar	ligar, liga, acender, on	Aciona os dois motores com rampa de aceleração.	Só no estado ARMADO
Desligar	desligar, desliga, apagar, off	Zera os dois canais na hora; a sessão continua montada.	LANÇANDO e PAUSADO
Pausar	pausa, pausar, espera	Suspende a contagem e para os motores.	LANÇANDO
Retomar	retoma, retomar, continua	Volta a sessão da etapa em que parou.	PAUSADO
Emergência	para, emergência, socorro	Interrompe tudo e bloqueia o sistema até o reinício.	PREPARANDO, ARMADO, LANÇANDO e PAUSADO

3.3 Travas de segurança

Voz é um canal assimétrico. Um "desliga" reconhecido por engano apenas para a máquina, que é o lado seguro. Um "liga" reconhecido por engano coloca uma máquina de 14 000 RPM em movimento. O Quadro 5 trata cada direção de acordo com esse risco.

Quadro 5 — Travas de segurança do canal de voz.

Trava	Como funciona
Ligar só no estado ARMADO	O comando de ligar por voz é recusado em qualquer estado que não seja ARMADO. Chegar lá exige checklist de preparação completo e aceite de risco registrado. A trava é o estado do sistema, não uma segunda fala, o que mantém o diálogo curto e a regra verificável.
Prioridade do desligar	Na validação da frase, o grupo de desligar é testado antes do de ligar. Uma frase ambígua que contenha os dois grupos resulta em desligar.
Desligar sempre disponível	Em LANÇANDO e em PAUSADO o comando de desligar é aceito sem nenhuma condição. Não existe estado ativo em que parar a máquina seja proibido.
Emergência não depende de voz	A parada de emergência física e local é a principal. A palavra de emergência por voz é uma segunda via, porque o canal de voz tem latência de rede e não atende à meta de 150 ms do Quadro 7.
Comando autenticado	Todo comando chega com o token compartilhado. Sem ele o ESP32 responde HTTP 401 e nenhum motor é acionado. No código atual o token viaja na query string, o que o deixa registrado no histórico do navegador e no log do servidor; movê-lo para um cabeçalho é correção prevista para a Entrega 2.
Termo ambíguo recusado	Frase sem termo-chave não gera requisição nenhuma. O sistema prefere não agir a agir errado.

3.4 Limitação declarada: a dependência de rede

O reconhecimento de fala do navegador não roda no aparelho. O Chrome envia o áudio para os servidores do Google e recebe o texto de volta, o que significa que **a transcrição exige internet**. O ESP32 cria uma rede própria por **softAP** sem rota para fora, então o celular conectado a ela só alcança o Google pelos próprios dados móveis.

Nos testes do grupo o controle por voz funciona porque o aparelho mantém o 4G ativo enquanto está na rede do ESP32, e o sistema operacional encaminha o tráfego de internet por ele. Isso é comportamento do celular, não do projeto. Numa quadra coberta, num clube com sinal ruim ou num aparelho sem plano de dados a voz para de funcionar, ainda que o restante da sessão continue, porque **/ligar** e **/desligar** trafegam na rede local.

Declaramos a limitação em vez de contorná-la no texto, porque ela tem duas saídas mapeadas para a Entrega 2. A primeira é trocar o reconhecimento remoto por um modelo embarcado no aparelho, como Vosk ou Whisper.cpp, que é o que atende ao RF21 sem depender de rede e é a solução tecnicamente correta para quadra. A segunda é tirar o ESP32 do modo Access Point e colocá-lo como cliente de uma rede que já tenha internet, o que resolve com uma linha de firmware mas passa a exigir infraestrutura de rede no local.

4 Fluxo de dados

A Figura 3 percorre o sistema na ordem que o enunciado pede: **entrada, processamento, saída e registro**. A entrada principal passa a ser o comando de voz.

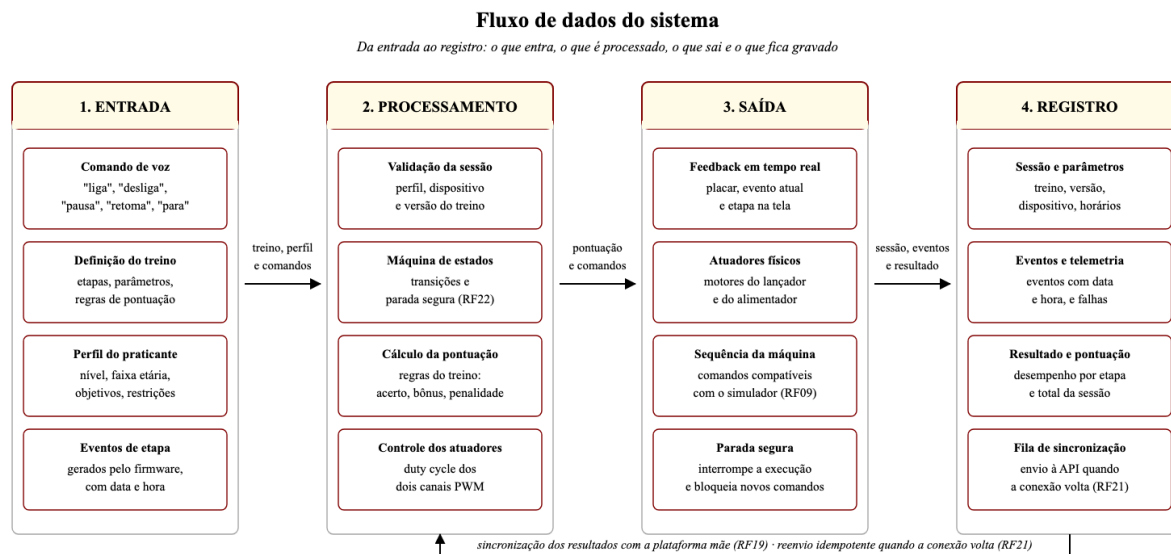


Figura 3 — Fluxo de dados do sistema. Fonte: Autoria própria.

Quadro 6 — O que cada etapa do fluxo recebe, processa, emite e registra.

Etapa	Entrada	Processamento	Saída	Registro
1. Entrada	Comando de voz transcrito, definição do treino pela API, perfil do praticante e eventos de etapa.	Validação do token, do estado da máquina, do perfil e da versão do treino.	Comando aceito ou recusado.	Comando, frase transcrita e horário.
2. Processamento	Comando validado e eventos com data e hora.	Máquina de estados, regras de pontuação e cálculo do duty cycle dos dois canais.	Pontuação acumulada e comandos aos atuadores.	Cada evento, com tipo, etapa e carimbo de tempo.
3. Saída	Pontuação e etapa atual.	Montagem do feedback e tradução da etapa em sinal de LED, buzzer e motor.	Placar na tela, sinalização em quadra e parada segura.	Eventos de falha e de parada segura.
4. Registro	Sessão encerrada, com eventos e pontuação.	Fechamento da pontuação e preparação do resumo e da telemetria.	Resumo para o praticante e envio à plataforma mãe.	Resultado local e fila de sincronização, com reenvio idempotente.

4.1 Metas de latência

O Quadro 7 fixa as metas por etapa. O critério foi a dinâmica do treino: o intervalo entre duas bolas é de poucos segundos, então um retorno acima de um quinto de segundo chega tarde para orientar o golpe seguinte.

Quadro 7 — Metas de latência para os eventos críticos.

Trecho medido	Meta	Por quê
Fala até o comando ser aceito	$\leq 1,5$ s	É o custo do canal de voz, dominado pela transcrição remota. Aceitável para ligar e desligar entre séries.
Evento de etapa até LED e buzzer	≤ 100 ms	É o retorno que o praticante percebe em quadra.
Evento de etapa até o placar do app	≤ 200 ms	O placar é consultado entre etapas, mas ainda precisa parecer imediato.
Comando aceito até os motores parados	≤ 150 ms	O trecho crítico. Depende do ESP32, não da rede, e é por isso que a emergência é física.
Encerramento até o resultado gravado localmente	≤ 1 s	O resumo precisa aparecer ainda em quadra, sem depender de rede.

A meta de 150 ms é a que manda no projeto, e ela é a razão de o comando de emergência ser tratado no firmware e não na API. Um comando que dependa de rede para chegar não é uma parada de emergência.

4.2 O caminho offline

O RF21 exige que o treino sincronizado possa ser executado sem conexão. A entrada passa a vir do armazenamento local, o processamento acontece no aparelho, a saída continua completa, porque o feedback depende do app e do dispositivo e não do servidor, e o registro vai para uma fila local. Duas regras evitam perda de dados: o reenvio é idempotente, para que o mesmo resultado enviado duas vezes não duplique a sessão, e o treino só fica disponível offline se foi baixado de forma explícita. O comando de voz é a única parte da sessão que não sobrevive sem rede, pelo motivo da Seção 3.4.

5 Requisitos funcionais e não funcionais

O Quadro 8 reproduz os 23 requisitos funcionais do enunciado com uma coluna de escopo. A coluna é nossa, não do enunciado: ela existe para que o documento sirva de instrumento de planejamento.

Quadro 8 — Requisitos funcionais e escopo no projeto.

ID	Canal	Funcionalidade	Escopo
RF01	Mobile/Site	Autenticação e perfis	MVP: login, logout e três perfis de acesso.
RF02	Mobile	Perfil do praticante	MVP: nível, faixa etária, objetivos e restrições.
RF03	Mobile	Catálogo de treinos	MVP: consulta e filtros por público, duração e dificuldade.
RF04	Mobile	Detalhes e preparação	MVP: regras, riscos e critérios de pontuação antes da sessão.
RF05	Mobile/IoT	Pareamento de dispositivo	MVP: descoberta, conexão e status dos sensores e atuadores.
RF06	Mobile	Execução da sessão	MVP: iniciar, pausar, retomar e encerrar, por voz e por toque.

ID	Canal	Funcionalidade	Escopo
RF07	IoT	Aquisição de sensores	MVP parcial: o microfone é o sensor principal; os eventos de etapa são gerados pelo firmware (Quadro 1).
RF08	IoT	Controle de atuadores	MVP: dois canais de motor por PWM, mais LED e buzzer.
RF09	IoT/API	Comandos da máquina ou simulador	MVP, núcleo do projeto: ligar e desligar por voz, com os parâmetros do exercício.
RF10	Mobile/IoT	Pontuação gamificada	MVP: pontos, bônus de sequência e penalidades por regras declaradas.
RF11	Mobile	Modos por público	MVP: modo adulto e modo criança, com linguagem e recompensas distintas.
RF12	Mobile	Resultado da sessão	MVP: resumo com desempenho por etapa, metas e eventos relevantes.
RF13	Mobile/Site	Histórico e indicadores	MVP: sessões por período e treino, com precisão e volume.
RF14	Site	Editor de treinos	MVP: etapas, regras, parâmetros e critérios de conclusão.
RF15	Site	Simulação e validação	MVP: simulação do treino antes da publicação.
RF16	Site	Versionamento e publicação	MVP: rascunho, versão publicada, autor e histórico.
RF17	Site	Gestão administrativa	MVP enxuto: usuários, dispositivos e categorias.
RF18	API	Consumo de treinos	MVP: contrato REST/JSON em OpenAPI, com versão do treino em toda resposta.
RF19	API	Envio de resultados	MVP: resumo da sessão e telemetria autorizada à plataforma mãe.
RF20	Site/API	Metadados de marketplace	MVP: autoria, categoria e licença registradas, sem cobrança.
RF21	Mobile/IoT	Operação offline	MVP parcial: a sessão roda offline, mas o comando de voz depende de rede (Seção 3.4).
RF22	Todos	Parada segura	MVP: emergência física e por voz, com estado absorvente até o reinício.
RF23	Todos	Auditoria e logs	MVP: comandos de voz, autenticação, falhas e parada segura.

5.1 Requisitos não funcionais

Qualidade não se entrega por declaração. O Quadro 9 converte cada requisito não funcional do enunciado em uma decisão concreta ou em uma meta verificável.

Quadro 9 — Requisitos não funcionais e como cada um se torna verificável.

Requisito	Como vira decisão verificável
Segurança	Autorização por perfil conferida na API; validação de entrada em toda rota; TLS no tráfego externo; segredos por variável de ambiente.
Cibersegurança do IoT	Token exigido nos endpoints de comando, com HTTP 401 na ausência; token guardado no cliente, nunca embutido na página servida.

Requisito	Como vira decisão verificável
Segurança física	O ligar por voz só é aceito no estado ARMADO; a emergência física é independente de rede e de voz; aceite de risco antes de armar; pinos zerados na inicialização.
Desempenho	Metas de latência do Quadro 7, medidas na Entrega 02 com o firmware em operação.
Disponibilidade e resiliência	Execução offline do treino sincronizado; fila local de resultados; reenvio idempotente; o desligar por voz continua funcionando sem internet.
Interoperabilidade	Contrato OpenAPI versionado; JSON padronizado; versão do treino em toda resposta que o referencia.
Usabilidade e acessibilidade	A voz é a interface principal, o que atende mãos ocupadas e sol na tela; nenhum evento sinalizado só por som ou só por cor; alternativa em tabela para todo gráfico.
Privacidade e LGPD	O áudio da fala não é armazenado pelo sistema, só o texto transcrito e o comando; coleta mínima; consentimento do responsável para menores.
Manutenibilidade	Arquitetura em camadas com contrato explícito; troca do motor de reconhecimento restrita ao app; tipos compartilhados entre app, site e API.
Escalabilidade	API sem estado, para permitir mais de uma instância; telemetria separada dos dados transacionais.
Observabilidade	Logs estruturados com identificador de sessão, correlacionando comando de voz, dispositivo e API.
Compatibilidade	Navegadores atuais no site; versões mínimas de Android e iOS fixadas; ESP32 como plataforma embarcada.

6 Máquina de estados do sistema

A Figura 4 modela o comportamento do sistema. O controle por voz aparece no alfabeto de eventos, e o estado **ARMADO** é a peça central: ele existe para que o comando de ligar por voz tenha uma porta única e verificável. O estado **S** é absorvente, então uma vez acionada a emergência nenhuma entrada tira o sistema de lá.

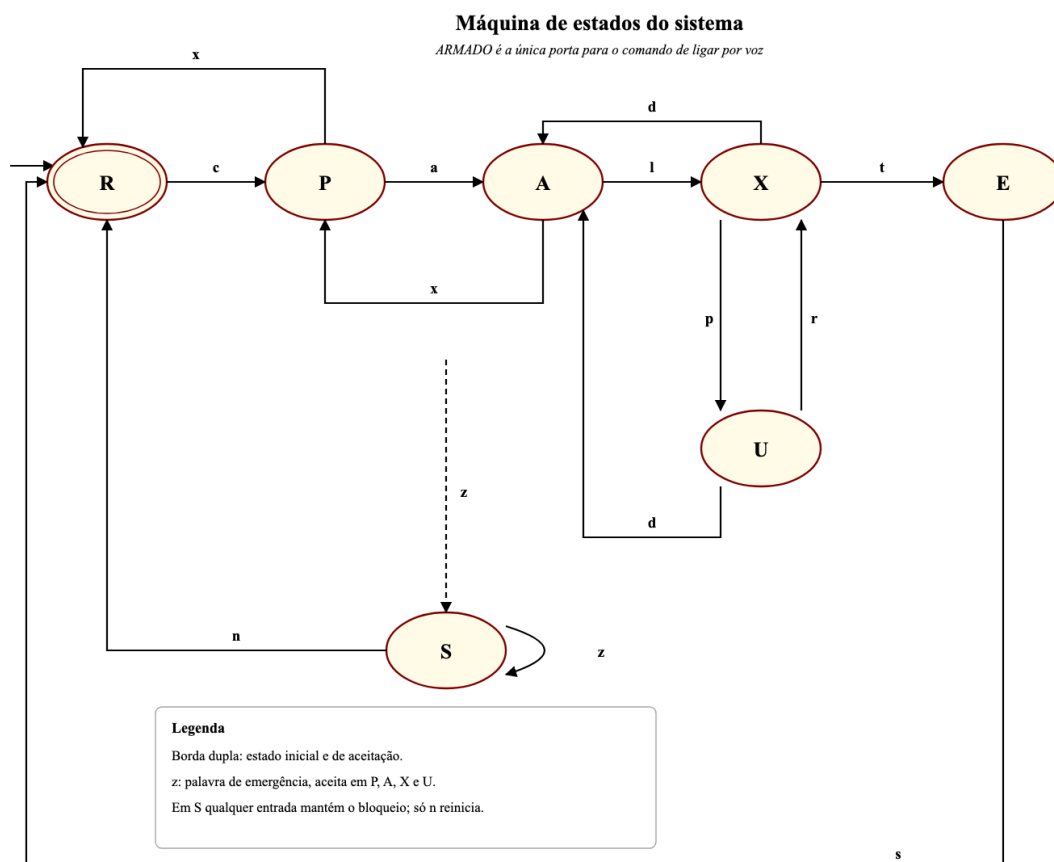


Figura 4 — Máquina de estados do sistema. Fonte: Autoria própria.

6.1 Estados

Quadro 10 — Estados da máquina do sistema.

Símbolo	Estado	O que representa
R	REPOUSO	Estado inicial e de aceitação. Máquina energizada, motores parados, nenhum dispositivo pareado.
P	PREPARANDO	Dispositivo conectado e sessão em montagem. O praticante percorre o checklist e registra o aceite de risco.
A	ARMADO	Checklist completo e risco aceito. É o único estado em que o comando de ligar por voz é aceito.
X	LANÇANDO	Motores girando e treino em execução. Os eventos de etapa chegam com data e hora e a pontuação acumula.
U	PAUSADO	Sessão suspensa pelo praticante. O tempo para de contar, o placar é preservado e os motores ficam parados.
E	ENCERRANDO	Etapas concluídas. O sistema fecha a pontuação e grava o resultado localmente.
S	PARADA SEGURA	Estado absorvente. A máquina está bloqueada e nenhum comando é aceito até o reinício explícito.

Quadro 11 — Função de transição da máquina do sistema.

O alfabeto de entrada é formado pelos comandos de voz já validados e pelos eventos de sessão: **c** (dispositivo conectado), **a** (checklist completo e risco aceito), **l** (liga), **d** (desliga), **p** (pausa), **r** (retoma), **z** (emergência), **t** (etapas concluídas), **s** (sincronização), **n** (reinício) e **x** (cancelar).

Estado atual	Evento	Próximo	O que o evento significa
R	c	P	Dispositivo conectado: o app parecia e recebe o status dos sensores.
P	a	A	Checklist de preparação completo e aceite de risco registrado.
P	x	R	Preparação abandonada antes de armar; nada foi acionado.
A	l	X	Comando de ligar por voz aceito; a rampa de aceleração começa.
A	x	P	Sistema desarmado; volta a preparar sem ter acionado os motores.
X	d	A	Comando de desligar por voz: os motores param e o sistema segue armado.
X	p	U	Comando de pausa: a contagem para e os motores param.
U	r	X	Comando de retomar: a sessão volta da etapa em que parou.
U	d	A	Comando de desligar com a sessão pausada.
X	t	E	A última etapa atingiu o critério de conclusão.
E	s	R	Resultado sincronizado ou colocado na fila; a sessão termina.
S	n	R	Reinício explícito, com confirmação dupla. É a única saída de S.
P, A, X, U	z	S	Palavra de emergência: os motores param e a máquina é bloqueada.
S	z	S	Absorvente: qualquer nova entrada mantém o sistema bloqueado.

6.2 Regras de funcionamento

- **O comando de ligar por voz só é aceito em ARMADO.** Em qualquer outro estado ele é recusado, e é isso que impede que a fala de um terceiro coloque a máquina em movimento.
- **O comando de desligar é aceito em LANÇANDO e em PAUSADO sem nenhuma condição.** Não existe estado ativo em que parar a máquina seja proibido.
- **Em S, nenhum comando chega à máquina.** Nem ligar, nem desligar, nem retomar; a única entrada com efeito é o reinício explícito.
- **Motores parados em R, P, A, U e S.** Só LANÇANDO aciona os atuadores.
- **A emergência é física e local.** A palavra de emergência por voz é a segunda via, porque o canal de voz não atende à meta de 150 ms.
- **O resultado é gravado localmente antes de qualquer tentativa de envio.**

6.3 Relação com a modelagem de Teoria da Computação

Esta máquina não substitui os autômatos M1 e M2 da entrega de Teoria da Computação, que descrevem o controle por voz do protótipo isolado. Ela descreve o sistema completo, em que o comando de voz é uma entrada entre outras e precisa conviver com preparação, pontuação e sincronização. As duas modelagens chegam à mesma decisão. Em M2 o estado DESLIGAR é absorvente e sobrepõe-se a qualquer comando de ligar na mesma frase; aqui S é absorvente e o desligar tem prioridade na validação. Diante de ambiguidade, o sistema escolhe o estado mais seguro, e essa escolha aparece tanto no autômato quanto na trava de segurança.

7 Product Backlog

O backlog está estruturado como Product Backlog do Scrum: **5 épicos** que agrupam **21 histórias de usuário**, cada uma com prioridade, estimativa em story points e critério de aceite verificável. O épico de controle por voz vem primeiro e concentra a Sprint 1, porque é o núcleo do projeto.

A escala de pontos é a sequência de Fibonacci (1, 2, 3, 5, 8, 13), usada para estimar **esforço relativo**, não prazo. A prioridade segue uma leitura simples de MoSCoW: **Alta** é o que precisa existir para o núcleo funcionar; **Média** é o que completa o produto.

7.1 Épicos

Quadro 12 — Épicos do Product Backlog.

ID	Épico	Objetivo
E1	Controle por voz	O núcleo do projeto: reconhecer a fala, validar o comando e acionar a máquina.
E2	Dispositivo IoT e acionamento	Parear, acionar os motores com segurança e responder o estado.
E3	Sessão de treino	Preparar, executar, pontuar e encerrar a sessão com parada segura.
E4	Contas, perfis e treinos	Entrada no sistema, perfil do praticante e autoria de treinos.
E5	API, offline e implantação	Contrato OpenAPI, integração com a plataforma mãe e sistema no ar.

7.2 Histórias de usuário

As histórias estão agrupadas por épico. A coluna de critério de aceite é o que a equipe verifica antes de considerar a história pronta.

Épico E1 — Controle por voz · 6 histórias · 36 pontos.

ID	História de usuário	Prio.	Pts	RF	Critério de aceite
US01	Como praticante, quero que o sistema entenda a minha fala em português, para comandar a máquina sem tocar em nada.	Alta	8	Núcleo	Reconhecimento contínuo em pt-BR; a frase transcrita aparece na tela para conferência.
US02	Como praticante, quero que o comando seja reconhecido dentro da frase, para não precisar falar robotizado.	Alta	5	Núcleo	O grupo de desligar é testado antes do de ligar, porque "desligar" contém "ligar"; frase sem termo-chave não gera requisição.
US03	Como praticante, quero ligar e desligar os motores falando, para controlar o disparo de onde eu estiver.	Alta	8	RF09	"Liga" aciona os dois motores com rampa; "desliga" para na hora; o estado volta para a tela.
US04	Como praticante, quero pausar e retomar o treino por voz, para descansar sem largar a raquete.	Alta	5	RF06	"Pausa" suspende a contagem e para os motores; "retoma" volta da mesma etapa.
US05	Como praticante, quero uma palavra de emergência que bloqueie a máquina, para parar tudo em caso de risco.	Alta	5	RF22	"Para" leva ao estado absorvente em qualquer fase ativa; só o reinício explícito libera o sistema.
US06	Como praticante, quero ver o que o sistema entendeu, para confiar que o comando certo foi aceito.	Média	5	RF23	Cada comando aceito ou recusado gera uma linha no terminal da página, com data e hora.

Épico E2 — Dispositivo IoT e acionamento · 4 histórias · 29 pontos.

ID	História de usuário	Prio.	Pts	RF	Critério de aceite
US07	Como praticante, quero conectar o celular ao dispositivo e ver o status dele, para saber que está pronto.	Alta	8	RF05	A página consulta /status ao carregar e mostra LIGADO ou DESLIGADO.
US08	Como praticante, quero que os motores acelerem suavemente ao ligar e parem na hora ao desligar.	Alta	8	RF08	Rampa de 51 passos de 15 ms; duty de 100/255 no lançador e 25/255 no alimentador; desligar zera os dois pinos.
US09	Como operador, quero que só comandos autenticados acionem os motores, para ninguém acionar a máquina por acidente.	Alta	5	RNF ciber segurança	Requisição sem token válido recebe HTTP 401 e não aciona motor; o token não vai embutido na página.
US10	Como praticante, quero que o dispositivo execute a sequência do treino no simulador, para a sessão andar sozinha.	Média	8	RF09	Parâmetros da etapa chegam ao dispositivo; o que a camada de adaptação não cobre está declarado.

Épico E3 — Sessão de treino · 4 histórias · 26 pontos.

ID	História de usuário	Prio.	Pts	RF	Critério de aceite
US11	Como praticante, quero preparar a sessão com um checklist e aceite de risco, para armar o sistema com segurança.	Alta	5	RF04	Sem checklist completo e aceite marcado o sistema não chega ao estado ARMADO, e o comando de ligar por voz é recusado.
US12	Como praticante, quero acompanhar etapa, tempo e placar em tempo real durante o treino.	Alta	8	RF06 / RF10	Feedback visual, sonoro e tátil a cada evento; nenhum evento depende só de som ou só de cor.
US13	Como praticante, quero que a parada de emergência bloqueie a máquina até eu reiniciar, para não haver acionamento acidental.	Alta	5	RF22	O estado é absorvente; o reinício exige confirmação dupla e a emergência física continua funcionando sem rede.
US14	Como praticante, quero ver o resumo da sessão e o meu histórico, para acompanhar a evolução.	Média	8	RF12 / RF13	Resumo com desempenho por etapa; histórico filtrável por período e por treino.

Épico E4 — Contas, perfis e treinos · 4 histórias · 28 pontos.

ID	História de usuário	Prio.	Pts	RF	Critério de aceite
US15	Como usuário, quero entrar e sair da minha conta com um perfil de acesso, para ver apenas o que me cabe.	Alta	5	RF01	Autorização conferida na API, não só escondida na tela; teste com perfil indevido devolve 403.
US16	Como praticante, quero registrar nível, faixa etária e restrições, para receber treinos adequados.	Média	5	RF02 / RF11	Faixa etária até 12 anos entra no modo criança, com consentimento do responsável.
US17	Como praticante, quero consultar o catálogo e ver as regras do treino antes de começar.	Alta	5	RF03 / RF04	A seção de pontuação é obrigatória e não fica atrás de um "saiba mais".
US18	Como autor, quero criar, versionar e publicar treinos no site.	Média	13	RF14 / RF16	Rascunho não aparece no catálogo; publicar cria versão imutável com autor e data.

Épico E5 — API, offline e implantação · 3 histórias · 29 pontos.

ID	História de usuário	Prio.	Pts	RF	Critério de aceite
US19	Como administrador, quero uma API com contrato OpenAPI que envie os resultados à plataforma mãe.	Alta	8	RF18 / RF19	Especificação publicada e coerente com os endpoints; falha de envio não bloqueia o fim da sessão.
US20	Como praticante, quero executar um treino sincronizado sem conexão e enviar o resultado depois.	Média	13	RF21	Sessão completa roda offline; reenvio idempotente não duplica a sessão no servidor.

ID	História de usuário	Prio.	Pts	RF	Critério de aceite
US21	Como equipe, quero implantar o sistema em ambiente acessível, para a demonstração na Semana FECAP.	Média	8	RNF observabilidade	URL pública responde ao fluxo completo; logs permitem correlacionar sessão, dispositivo e API.

7.3 Critérios de pronto

- O código está versionado no repositório e revisado por outro integrante.
- Os critérios de aceite foram verificados e o resultado está registrado.
- O requisito associado na coluna RF tem evidência de funcionamento, não só de implementação.
- Não há segredo, senha ou chave em texto claro no código entregue.

7.4 Planejamento das sprints

As 21 histórias somam 148 pontos, distribuídos em três sprints de duas semanas mais uma sprint zero de preparação. A Sprint 1 é inteiramente dedicada ao núcleo de voz e ao acionamento. A Sprint 3 coincide com o fechamento da Entrega 02.

Quadro 13 — Distribuição das histórias por sprint.

Sprint	Foco	Pontos	Histórias
Sprint 0	Preparação: repositório, ambientes e contrato inicial da API.	—	—
Sprint 1	Núcleo de voz e acionamento: reconhecer, validar, ligar e desligar com segurança.	42	US01 a US03, US07 a US09
Sprint 2	Sessão e comandos completos: pausar, retomar, emergência e preparação.	46	US04 a US06, US10 a US13, US15
Sprint 3	Plataforma e entrega: resultado, histórico, autoria, offline e implantação.	60	US14, US16 a US21

Se a velocidade real da equipe ficar abaixo do previsto, o corte natural é a Sprint 3: as histórias de prioridade Média são as que não bloqueiam o núcleo de voz nem o cenário de aceitação.

7.5 Importação no GitHub Projects

O backlog também é mantido no GitHub Projects do repositório, a ferramenta de gestão citada pelo enunciado. Para que o quadro e este documento não diverjam, as histórias são geradas a partir do mesmo arquivo-fonte que alimenta o CSV de importação em `docs/entregas/pi-iot/backlog.csv`, com as colunas Title, Epic, Priority, Story

Points, RF, Sprint, Status, Description e Acceptance Criteria.

8 Uso de inteligência artificial

Por transparência acadêmica, declaramos as ferramentas de inteligência artificial usadas na produção deste documento. Todo o material passou por revisão humana do grupo, e as decisões de arquitetura, escopo e priorização são de autoria da equipe.

- **Redação e diagramação deste relatório, com revisão humana** — Claude Sonnet 5. Os diagramas das Figuras 1 a 4 foram gerados por script próprio do grupo (`scripts/generate_pi_diagrams.py`), a partir de especificação da equipe.
- **Apoio à estruturação do Product Backlog, com revisão humana** — Claude Sonnet 5. As histórias foram derivadas dos requisitos RF01 a RF23 do enunciado e revisadas pelo grupo.
- **Geração de código do firmware do ESP32** — Gemini 3.8 Flash, conforme já declarado na entrega de Sistemas Embarcados e Robótica.
- **Geração das imagens do circuito** — Deepseek V4.1 Flash, conforme já declarado na entrega de Sistemas Embarcados e Robótica.